Improving Custom Model Performance

Improve used-car price prediction through error analysis, feature engineering, preprocessing pipelines, and controlled hyperparameter search across scikit-learn, Keras, and managed services.

Original study notes based on Gwendolyn Stripling and Michael Abel, First edition, 2023. Book pp. 249–288. Source APIs, services, experiments, and model comparisons reflect the book's publication period. These are explanatory summaries, not verbatim reproductions or a claim that every source detail is retained.

Baseline + residuals → Engineer features → Tune on validation → Compare fairly → Test once, then deploy

Detailed Notes

Used-car business case and baseline

Source: book pp. 249–255 / PDF p. 265

Start from an existing linear model and identify what "better" means for auction-price decisions. Inspect target and feature distributions, split sizes, and evaluation metrics. Residuals plotted against price, vehicle age, mileage, make, or category can reveal nonlinearities, heteroscedasticity, and systematic subgroup errors.

Keep train/validation/test roles distinct. The test set is for final estimation, not repeated feature decisions. A cleaned supplied dataset still deserves independent schema and leakage checks.

Feature engineering and preprocessing pipelines

Source: book pp. 256–260 / PDF p. 272

Use domain understanding to derive age from year, consider transformations for skewed price/mileage, manage rare categories, and handle missing values. Add interactions or nonlinear features only when validation supports them.

Fit imputers, scalers, encoders, and feature selectors inside a scikit-learn Pipeline so cross-validation learns preprocessing only from each training fold. This prevents leakage and makes prediction inputs reproducible. Keras preprocessing layers can be adapted on training data and saved with the model.

Hyperparameter tuning in scikit-learn and Keras

Source: book pp. 261–275 / PDF p. 277

Hyperparameters configure learning rather than being learned weights: regularization strength, tree depth, layer width, dropout, learning rate, and batch size are examples. Search them on validation/cross-validation data, using a justified metric and reasonable budget.

Repeated tuning overfits the validation set too. Record trials, ranges, seeds, runtime, and resource limits. Keras trials need comparable training budgets and early-stopping rules. Prefer randomized/Bayesian search when exhaustive grids are too costly; final test once after choices are fixed.

BigQuery ML, Vertex AI, and other tuning services

Source: book pp. 276–288 / PDF p. 292

BigQuery ML can transform data and tune supported models through SQL. The TRANSFORM clause can preserve preprocessing with the model. Vertex AI and other cloud services distribute training/search; configure data paths, region, service accounts, machine types, budget, and output locations.

Large searches spend money and can congest quotas. Use a smaller diagnostic search before a full run. Provider-specific interfaces and pricing age quickly; retain the concepts (search space, objective, validation, budget, reproducibility) and consult current documentation for exact console steps.

Revision Check

1. What residual pattern would motivate a feature transformation?
2. Where should preprocessing be fitted during cross-validation?
3. Why keep a final test set untouched?
4. What costs and metadata should a tuning job track?

Improve through diagnosis, not random complexity. A reproducible pipeline and honest validation matter as much as the chosen algorithm.

Source links refer to the supplied PDF. Every contents-page topic is mapped in coverage.md. Consult the book for complete implementations, figures, exercises, citations, and experiment evidence.